

Лабораторная работа №2

Исследование протокола TCP и алгоритма управления очередью RED

Чемоданова Ангелина Александровна

16 февраля 2025

Российский университет дружбы народов имени Патриса Лумумбы, Москва, Россия

Информация

- Чемоданова Ангелина Александровна
- Студентка НФИбд02-22
- Российский университет дружбы народов имени Патриса Лумумбы
- 1132226443@pfur.ru
- <https://github.com/aachemodanova>



Исследовать протокол TCP и алгоритм управления очередью RED.

1. Рассмотреть пример с дисциплиной RED.
2. Изменить в модели на узле s1 тип протокола TCP с Reno на NewReno, затем на Vegas.
Сравнить и пояснить результаты.
3. Внести изменения при отображении окон с графиками (изменить цвет фона, цвет траекторий, подписи к осям, подпись траектории в легенде).

Постановка задачи. Описание моделируемой сети: – сеть состоит из 6 узлов; – между всеми узлами установлено дуплексное соединение с различными пропускной способностью и задержкой 10 мс; – узел r1 использует очередь с дисциплиной RED для накопления пакетов, максимальный размер которой составляет 25; – TCP-источники на узлах s1 и s2 подключаются к TCP-приёмнику на узле s3; – генераторы трафика FTP прикреплены к TCP-агентам.

Скрипт модели с дисциплиной RED

```
# создание объекта Simulator
set ns [new Simulator]

# открытие на запись файла out.nam для визуализатора nam
set nf [open out.nam w]

# все результаты моделирования будут записаны в переменную nf
$ns namtrace-all $nf

# открытие на запись файла трассировки out.tr
# для регистрации всех событий
set f [open out.tr w]

# все регистрируемые события будут записаны в переменную f
$ns trace-all $f

# процедура finish закрывает файлы трассировки
# и запускает визуализатор nam
# Процедура finish:
proc finish {} {
    global tchan
    # подключение кода AWK:
    set awkCode {
        {
            if ($1 == "q" && NF>2) {
                print $2, $3 >> "temp.q";
                set end $2
            }
            else if ($1 == "a" && NF>2)
                print $2, $3 >> "temp.a";
        }
    }
    set f [open temp.queue w]
    puts $f "TitleText: red"
    puts $f "Device: Postscript"
    if { [info exists tchan] } {
        close $tchan
    }
    exec rm -f temp.q temp.a
    exec touch temp.a temp.q
    exec awk $awkCode all.q
    puts $f "\nqueue"
    exec cat temp.q >> $f
    puts $f "\nave_queue"
    exec cat temp.a >> $f
    close $f
    # Запуск xgraph с графиками окна TCP и очереди:
    exec xgraph -bb -tk -x time -t "TCP Reno Cwnd" WindowVsTimeReno &
    exec xgraph -bb -tk -x time -y queue temp.queue &
    exit 0
}

proc plotWindow (tcpSource file) {
    global ns
    set time 0.01
    set now [$ns now]
    set cwnd [$tcpSource set cwnd]
    puts $file "Show $cwnd"
    $ns at [expr $now+$time] "plotWindow $tcpSource $file"
}

# Удалить сети:
set B 5
```

Рис. 1: Скрипт модели с дисциплиной RED

Скрипт модели с дисциплиной RED

```
# Узлы сети:
set N 5
for {set i 1} {$i < $N} {incr i} {
    set node_($i) [$ns node]
}
set node_(r1) [$ns node]
set node_(r2) [$ns node]

# Соединения:
$ns duplex-link $node_(s1) $node_(r1) 10Mb 2ms DropTail
$ns duplex-link $node_(s2) $node_(r1) 10Mb 3ms DropTail
$ns duplex-link $node_(r1) $node_(r2) 1.5Mb 20ms RED
$ns queue-limit $node_(r1) $node_(r2) 25
$ns queue-limit $node_(r2) $node_(r1) 25
$ns duplex-link $node_(s3) $node_(r2) 10Mb 4ms DropTail
$ns duplex-link $node_(s4) $node_(r2) 10Mb 5ms DropTail

# Агенты и приложения:
set tcp1 [$ns create-connection TCP/Reno $node_(s1) TCPSink $node_(s3) 0]
$tcp1 set window 15
set tcp2 [$ns create-connection TCP/Reno $node_(s2) TCPSink $node_(s3) 1]
$tcp2 set window 15
set ftp1 [$tcp1 attach-source FTP]
set ftp2 [$tcp2 attach-source FTP]

# Мониторинг размера окна TCP:
set windowVsTime [open WindowVsTimeReno w]
set qmon [$ns monitor-queue $node_(r1) $node_(r2) [open qm.out w] 0.1];
[$ns link $node_(r1) $node_(r2)] queue-sample-timeout;

# Мониторинг очереди:
set redq [$ns link $node_(r1) $node_(r2)] queue]
set tchan [open all.q w]
$redq trace curq_
$redq trace ave
$redq attach $tchan_

# Добавление ат-событий:
$ns at 0.0 "$ftp1 start"
$ns at 1.1 "plotWindow $tcp1 $windowVsTime"
$ns at 3.0 "$ftp2 start"
$ns at 10 "finish"

# запуск модели
$ns run
```

Рис. 2: Скрипт модели с дисциплиной RED

График динамики размера окна TCP(сверху) и график динамики длины очереди и средней длины очереди(снизу) при типе протокола TCP Reno на узле s1.

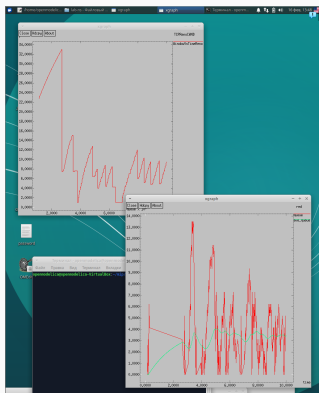


Рис. 3: График динамики размера окна TCP(сверху) и график динамики длины очереди и средней длины очереди(снизу) при типе протокола TCP Reno на узле s1

Изменения в модели на узле s1 типа протокола TCP с Reno на NewReno, затем на Vegas.

Скрипт изменений на узле s1 типа протокола TCP с Reno на Newreno.

```
# Агенты и приложения:  
set tcp1 [$ns create-connection TCP/Newreno $node_(s1) TCPSink $node_(s3) 0]  
$tcp1 set window_ 15  
set tcp2 [$ns create-connection TCP/Reno $node_(s2) TCPSink $node_(s3) 1]  
$tcp2 set window_ 15  
set ftp1 [$tcp1 attach-source FTP]  
set ftp2 [$tcp2 attach-source FTP]
```

Рис. 4: Скрипт изменений на узле s1 типа протокола TCP с Reno на Newreno

График динамики размера окна TCP(снизу) и график динамики длины очереди и средней длины очереди(сверху) при типе протокола TCP NewReno на узле s1.

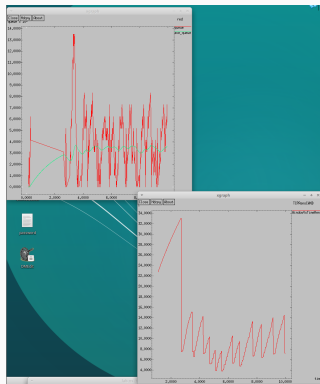


Рис. 5: График динамики размера окна TCP(снизу) и график динамики длины очереди и средней длины очереди(сверху) при типе протокола TCP NewReno на узле s1

Скрипт изменений на узле s1 типа протокола TCP с Reno на Vegas.

```
# Агенты и приложения:  
set tcp1 [$ns create-connection TCP/Vegas $node_(s1) TCPSink $node_(s3) 0]  
$tcp1 set window_ 15  
set tcp2 [$ns create-connection TCP/Reno $node_(s2) TCPSink $node_(s3) 1]  
$tcp2 set window_ 15  
set ftp1 [$tcp1 attach-source FTP]  
set ftp2 [$tcp2 attach-source FTP]
```

Рис. 6: Скрипт изменений на узле s1 типа протокола TCP с Reno на Vegas

График динамики размера окна TCP(снизу) и график динамики длины очереди и средней длины очереди(сверху) при типе протокола TCP Vegas на узле s1.

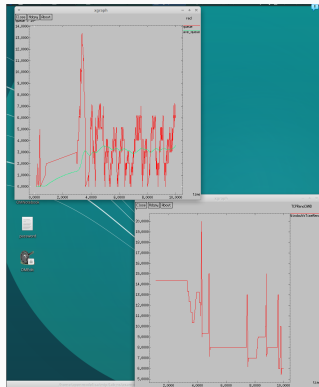


Рис. 7: График динамики размера окна TCP(снизу) и график динамики длины очереди и средней длины очереди(сверху) при типе протокола TCP Vegas на узле s1

Изменения при отображении окон с графиками (изменить цвет фона, цвет траекторий, подписи к осям, подпись траектории в легенде).

Изменение процедуры finish.

```
proc finish {} {  
  global tchan_  
  # подключение кода AWK:  
  set awkCode {  
    {  
      if ($1 == "Q" && NF>2) {  
        print $2, $3 >> "temp.q";  
        set end $2  
      }  
      else if ($1 == "a" && NF>2)  
        print $2, $3 >> "temp.a";  
    }  
  }  
  set f [open temp.queue w]  
  puts $f "TitleText: RED"  
  puts $f "Device: Postscript"  
  puts $f "0.Color: Orange"  
  puts $f "1.Color: Purple"  
  if { [info exists tchan_] } {  
    close $tchan_  
  }  
  exec rm -f temp.q temp.a  
  exec touch temp.a temp.q  
  exec awk $awkCode all.q  
  puts $f "\nQ"  
  exec cat temp.q >@ $f  
  puts $f "\n\"avg_Q"  
  exec cat temp.a >@ $f  
  close $f  
  # Запуск xgraph с графиками окна TCP и очереди:  
  exec xgraph -bg white -bb -tk -x time -y Razner "TCP Reno Cwnd" WindowVsTimeReno &  
  exec xgraph -bg white -bb -tk -x time -y QUEUE temp.queue &  
  exit 0  
}
```

Рис. 8: Изменение процедуры finish

Изменение мониторинга размера окна TCP.

```
# Мониторинг размера окна TCP:
set windowVsTime [open WindowVsTimeReno w]
puts $windowVsTime "0.Color: Black"
puts $windowVsTime \"dinamica_raz_okna"
set qmon [$ns monitor-queue $node_(r1) $node_(r2) [open qm.out w] 0.1];
[$ns link $node_(r1) $node_(r2)] queue-sample-timeout;
```

Рис. 9: Изменение мониторинга размера окна TCP

Результаты изменений отображения окон с графиками.

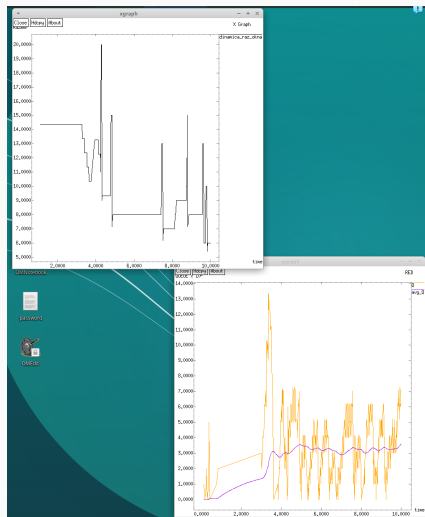


Рис. 10: Результаты изменений отображения окон с графиками

Мы исследовали протокол TCP и алгоритм управления очередью RED.